

탁구게임기의 코드 구현 및 실행

2021-17385

산업공학과 김예원

먼저 탁구게임기의 코드를 파이썬으로 재구현해보았다. 방학 때 탁구게임기를 워크샵에서 실행해본 적이 있는데, 당시에는 코드를 복사, 붙여넣기만 했지 그 논리를 이해하지 못했다. 이번 기회로 한줄 한줄 작성하며 그 의미를 이해하려고 노력했다. 특히 공이 패들에 닿았는지 확인해 방향을 바꿔주는 함수의 논리를 이해하는 것이 재밌었다. 그리고 파이썬 코드에서는 x축 방향과 y축 방향을 나눠서 순서대로 처리하였다. 이를 Verilog 상에서 구현한다면 동시에 병렬 회로로 실행할 수 있다는 것이 놀라웠다. 하드웨어와 소프트웨어 구현의 차이점을 뚜렷하게 느낄 수 있는 지점이었다.

```
if xPos == (WIDTH - PADOLE_W - BALL_SIZE) and yPos < (yPaddle + PADOLE_H) and yPos > yPaddle:
    dx = False
elif xPos == (TABLE_X + TABLE_W):
    dx = True
#WIDTH - PADOLE_W - BALL_SIZE = 128 - 5 - 8 = 115. "공이 막 패들 앞에 도달한 순간"의 x좌표" 한 픽셀씩 움직이는 구조라서 ==만으로도 충분함.
#y조건 - "공의 y좌표가 패들의 위쪽 끝보다 아래(크고), 패들의 아래쪽 끝보다 위(작다)" => 공이 패들 운동 범위에 있음.
if 패들에 닿았는가 -> false, elif왼쪽벽에 닿았는가 -> true
#true->false로 바뀌면 dx 변경된 것. 다음 코드를 통해 방향을 반대로 변경하게 됨.
#반대로 왼쪽 벽에 닿으면 true로 유지되니까, 공이 멈추지 않고 계속 진행해 게임오버로 이어짐.

xPos = xPos + 1 if dx else xPos - 1
#실제 위치를 이동하는 코드. 위에서 if절에 따라 방향이 결정되고 난 이후에 이동이 일어난. 새로운 방향으로 한 칸 이동!!!!

# 위아래 벽 뚫김 처리 (x축과 동일하게 y축에서도 실행)
if yPos == (HEIGHT - BALL_SIZE):
    dy = False
elif yPos == TABLE_Y:
    dy = True

yPos = yPos + 1 if dy else yPos - 1
#왼쪽은 x좌표, y좌표, y가 나중에 처리됨. 소프트웨어라서 순서를 정해야 함.
#만약 베릴로그로 된다면 x와 y 계산을 병렬로 돌릴 수 있다!!! (보고서에 넣기)
```

원본과 동일한 조건을 재현하기 위해 코드에서 위치 리셋 코드를 지워보았다. 공이 벽에 닿아 게임이 끝나면 score만 초기화되고, 공의 위치나 방향은 리셋되지 않아 공이 화면 밖으로 사라졌다. 이어 원본에 있던 xPos>SCREEN_WIDTH일 때 되돌리는 안전장치를 추가하니, xPos는 고정된 시작 위치로 복구됐지만 yPos와 방향은 여전히 직전 라운드 값을 그대로 이어받는 상태가 재현됐다. 마지막으로 xPos, yPos = TABLE_X + TABLE_W, TABLE_Y를 명시적으로 추가하자 공이 매년 동일한 위치에서 시작하는 걸 확인했다. 특히 게임기를 구현할 때, 리셋의 조건, 리셋 변수들을 빠짐없이 확인하는 것이 중요함을 알 수 있었다.

마지막으로 직접 보드에 코드를 실행해보았다. 아두이노에 코드를 업로드하자, 원래 켜져있던 오실로스코프 화면이 탁구게임기 화면으로 바뀌어 실행됨을 확인했다. 공이 움직이고 패들이 조작에 의해 이동하는 것을 보니 파이썬으로 구현하며 이해했던 로직이 실제 아두이노 보드에서 그대로 재현되는 것을 확인하는 과정 자체가 즐거웠다. 처음엔 막막하게 느껴졌지만, 막힐 때마다 하나씩 해결하고 그 결과물을 보니 뿌듯하고 재밌었다. 앞으로 나의 칩을 구현하는 것이 기대가 된다.

